

Contents

Project 2: Extending Apache Roller with Advanced Features and Design Patterns	2
CS6.401 - Software Engineering	2
1. Introduction	3
2. System Overview	3
3. Feature Implementations	4
Task 1A: Stars	4
Overview	4
Requirements Addressed	4
Design Patterns Used	4
Changes to Existing Code	4
New Files Created	5
Key Implementation Details	5
Task 1B: Trending Blogs	6
Overview	6
Requirements Addressed	6
Design Pattern: Aggregate Query Optimization (via Named Queries)	6
Task 2: Transforming Feeds	7
Overview	7
Requirements Addressed	7
Design Patterns Used	7
Comparison with Existing Plugin System	8
Changes to Existing Code	8
New Files Created	8
Key Implementation Details	9
Configuration	10
How to Add a New Step	10
Task 3: Bug Reporting System	11
1. Feature Description	11
2. Implementation	11
3. Assumptions / Constraints	12
4. Files Modified / Added	13
5. Design Patterns Used	14
6. Rationale Behind Pattern Selection	15
7. User Flow	15
8. UML Diagram	16
Task 4: Admin Dashboard	16
Overview	16
Requirements Addressed	16
Design Patterns Used	16
Metrics	17
Changes to Existing Code	18
New Files Created	18
Key Implementation Details	19

How to Use	19
Task 5: Web Translation and Caching	19
1. Feature Description	19
2. Implementation	20
3. Assumptions / Constraints	21
4. Files Modified / Added	22
5. Design Patterns Used	22
6. Rationale Behind Pattern Selection	23
7. User Flow	23
8. UML Diagram	24
Task 6: Community Pulse Dashboard	24
Overview	24
Requirements Addressed	24
Design Patterns Used	24
6A: Discussion Overview Indicators	25
6B: Conversation Breakdown Methods	26
Changes to Existing Code	26
New Files Created	27
Configuration	27
How to Use	27
UML Diagrams	28
Design Patterns Summary	28
UML Diagrams	29
Task 1A/1B — User Highlights	29
Task 2 — Transforming Feeds	29
Task 4 — Admin Dashboard	29
Testing	29
Task 1 — Unit Tests (10 POJO tests passing + 1 integration test)	29
Task 2 — Unit Tests (59 total, all passing)	30
Task 4 — Unit Tests (22 total, all passing)	30
Task 6 — Unit Tests (48 total, all passing)	31
How to Use	32
Feature 1: User Highlights (Stars & Trending)	32
Feature 2: Transforming Feeds Pipeline	32

Project 2: Extending Apache Roller with Advanced Features and Design Patterns

CS6.401 - Software Engineering

TEAM 33 Members - Aryan Mishra, 2024121001 - Aditya Nair, 2023111029 - Varun Patil, 2025204013 - Hardik Chadha, 2023111031 - Vijay Aravynthan, 2023111007

Repository: <https://github.com/serc-courses/project-1-team-33>

Branch: project2_33

1. Introduction

In this project, we extended the functionality of the Apache Roller blogging platform by implementing several new features aimed at improving user engagement, administrative monitoring, and content processing. The goal of this phase was to enhance the existing system while maintaining good software engineering practices such as modularity, extensibility, and maintainability.

The features implemented in this project include a **User Highlights system for starring blogs and identifying trending content**, a **content transformation pipeline for preprocessing blog posts**, a **bug reporting system with administrative management and notifications**, an **admin analytics dashboard**, **webpage translation with caching**, and a **community pulse dashboard that summarizes comment discussions**.

While implementing these features, emphasis was placed on applying appropriate **software design patterns** to ensure that the system remains flexible and easy to extend in the future. Each feature was designed as a modular component integrated with the existing Apache Roller architecture.

2. System Overview

Apache Roller is a Java-based blogging platform that allows users to create weblogs, publish blog posts, and interact with readers through comments and discussions. The system supports multiple users and blogs, making it suitable for collaborative blogging environments.

The core system includes modules responsible for **user management, weblog entry management, comment handling, and template-based blog rendering**. These components provide the foundation upon which additional features can be built.

In this project, we extended the existing system by adding new modules and services that integrate with the current architecture. The new components interact with the existing data models and service layers of Apache Roller while keeping the implementation modular and loosely coupled.

The implemented features focus on improving different aspects of the platform:

- increasing **user engagement** through starring and trending blogs
- enabling **automated content processing** through feed transformation pipelines
- allowing **users to report issues** and enabling administrators to manage them
- providing **administrative insights** through a dashboard with site metrics
- enabling **multilingual accessibility** through translation features
- summarizing **community discussions** through comment analysis tools

These additions enhance the usability and functionality of the platform while maintaining compatibility with the existing Roller system architecture.

3. Feature Implementations

Task 1A: Stars

Overview

Users can “star” (favourite) both weblogs and individual blog entries. Starred content is accessible from dedicated pages in the user’s navigation.

Requirements Addressed

- Users can visit blog pages and star them
- From their home page, users can view all starred weblogs, sorted by when the most recent blog post was made (most recently updated at top)
- Date and time of last post is displayed
- Users can also star individual blog posts
- Starred entries page uses pagination (not all in a single page)
- Hyperlinks in the home page navigate to starred content

Design Patterns Used

1. Data Access Object (DAO) Pattern **Where:** `StarManager` (interface) and `JPAStarManagerImpl` (JPA implementation)

Rationale: The DAO pattern separates the persistence logic from the business logic. `StarManager` defines a clean interface with methods like `saveWeblogStar()`, `getStarredWeblogs()`, and `getTrendingWeblogs()`. The JPA implementation (`JPAStarManagerImpl`) encapsulates all database interactions using named JPQL queries.

Quality attribute improvement: - *Maintainability* — Switching from JPA to another persistence framework requires only writing a new implementation of `StarManager`, no changes to actions or UI. - *Testability* — Actions can be tested against the interface using mocks.

Trade-off: Adds an extra layer of abstraction. For a small feature this could be over-engineering, but it is consistent with Roller’s existing architecture (e.g., `WeblogManager`, `UserManager`).

2. Dependency Injection (via Guice) **Where:** `JPAStarManagerImpl` is annotated with `@Singleton` and `@Inject`, bound in `JPAWebloggerModule`, and injected into `WebloggerImpl`.

Rationale: Follows Roller’s existing DI pattern. The `StarManager` is wired into the `Weblogger` facade so any component in the system can access it via `WebloggerFactory.getWeblogger().getStarManager()`.

Quality attribute improvement: - *Extensibility* — New managers can be added to the DI container without modifying existing code. - *Loose coupling* — Components depend on the `StarManager` interface, not the concrete implementation.

Trade-off: Requires understanding of Guice to add new bindings.

Changes to Existing Code

File	Change
Weblogger.java (interface)	Added <code>getStarManager()</code> method
WebloggerImpl.java	Added <code>starManager</code> field with constructor injection
JPAWebloggerModule.java	Added Guice binding: <code>StarManager -> JPASStarManagerImpl</code>
UtilitiesModel.java	Added <code>isWeblogStarred(weblogId)</code> and <code>isEntryStarred(entryId)</code> methods for Velocity templates
struts.xml	Added action mappings for <code>starWeblog</code> , <code>starEntry</code> , <code>starredWeblogs</code> , <code>starredEntries</code> , <code>trending</code>
tiles.xml	Added tile definitions for <code>.StarredWeblogs</code> , <code>.StarredEntries</code> , <code>.Trending</code>
ApplicationResources.properties	Added 15 i18n keys for star/trending UI text
createdb.vm	Added <code>user_weblog_star</code> and <code>user_entry_star</code> tables with indexes and foreign keys
Theme templates (4 themes)	Added star/unstar buttons on sidebar (weblogs) and permalink (entries)

New Files Created

File	Purpose
pojos/UserWeblogStar.java	JPA entity — user-to-weblog star with id, user, weblog, starredTime
pojos/UserEntryStar.java	JPA entity — user-to-entry star with id, user, entry, starredTime
pojos/UserWeblogStar.orm.xml	ORM mapping with 6 named JPQL queries
pojos/UserEntryStar.orm.xml	ORM mapping with 6 named JPQL queries
business/StarManager.java	Interface defining 14 star operations
business/jpa/JPASStarManagerImpl.java	JPA implementation with <code>@Singleton/@Inject</code>
ui/struts2/core/StarWeblogAction.java	Star/unstar weblog (prevents self-starring)
ui/struts2/core/StarEntryAction.java	Star/unstar entry (redirects back to permalink)
ui/struts2/core/StarredWeblogsAction.java	List user's starred weblogs
ui/struts2/core/StarredEntriesAction.java	List user's starred entries with pagination
ui/struts2/core/TrendingAction.java	Top 5 trending weblogs and entries
ui/struts2/pagers/StarredEntriesPager.java	Pagination support for starred entries list
WEB-INF/jsp/core/StarredWeblogs.jsp	JSP view for starred weblogs
WEB-INF/jsp/core/StarredEntries.jsp	JSP view for starred entries
WEB-INF/jsp/core/Trending.jsp	JSP view for trending content

Key Implementation Details

- **Sorted by most recent blog post:** The query `UserWeblogStar.getStarredWeblogsByUser` orders by `s.weblog.lastModified DESC` (not by when the user starred it), as required by the spec.
- **Pagination:** Starred entries use `StarredEntriesPager` with configurable page size (default 20). JPQL `setFirstResult()/setMaxResults()` handles DB-level pagination.

- **Idempotent starring:** Before creating a star, the system checks if one already exists. Starring twice does not create duplicates.
 - **Self-star prevention:** `StarWeblogAction` checks if the authenticated user owns the weblog and prevents self-starring.
 - **Star buttons across all themes:** Added to `basic`, `basicmobile`, `fauxcoly`, and `gaurav` themes using Velocity templates with `$utils.isWeblogStarred()` / `$utils.isEntryStarred()` checks.
-

Task 1B: Trending Blogs

Overview

Displays the top 5 trending weblogs and blog entries based on the number of stars (user favourites), using efficient aggregate queries.

Requirements Addressed

- Top 5 trending blogposts and blog pages based on number of stars
- Does NOT iterate over individual articles — uses optimized aggregate queries
- Efficient data fetching

Design Pattern: Aggregate Query Optimization (via Named Queries)

Where: `UserWeblogStar.orm.xml` and `UserEntryStar.orm.xml`

Rationale: The trending queries use `GROUP BY + COUNT + ORDER BY` in JPQL to compute star counts in a single database roundtrip, rather than loading all stars into memory and counting in Java.

Trending weblogs query:

```
SELECT s.weblog, COUNT(s) AS starCount
FROM UserWeblogStar s
GROUP BY s.weblog
ORDER BY starCount DESC
```

Trending entries query:

```
SELECT s.entry, COUNT(s) AS starCount
FROM UserEntryStar s
GROUP BY s.entry
ORDER BY starCount DESC
```

Quality attribute improvement: - *Performance* — $O(1)$ database queries regardless of the number of stars; the DB engine handles aggregation natively. - *Scalability* — Works efficiently even with millions of star records.

Trade-off: The queries return `Object[]` arrays (entity + count), which requires index-based access in the JSP/action layer rather than typed getters.

Task 2: Transforming Feeds

Overview

An admin-side entry processing pipeline that runs when blog entries are saved. It applies 5 configurable processing steps to entry content before persistence: profanity filtering, AI-powered summarization, sentiment analysis, reading time estimation, and auto tag generation.

Requirements Addressed

- Posted blogs go through 5 processing steps (exceeds the minimum 3)
- Steps include profanity filter, AI text summarization, sentiment analysis, reading time estimation, and tag generation
- Steps can be added/removed without affecting the remaining process
- Admin-side processing (not user-side)
- Tags added to the body of the blog (no separate storage needed)
- Summary stored in `entry.summary` field — Roller’s built-in `displayContent()` automatically shows summary + “Read More” on list pages, full text on permalink pages

Design Patterns Used

3. Chain of Responsibility Pattern Where: `EntryProcessingStep` (interface), `EntryProcessingPipeline` (orchestrator), and the five step implementations.

Rationale: The Chain of Responsibility pattern allows multiple processing steps to be applied sequentially to a blog entry, where each step can modify the entry independently. The pipeline maintains an ordered list of steps and executes each one, passing the same `WeblogEntry` object through. Steps do not know about each other — they only depend on the `EntryProcessingStep` interface.

How it enables add/remove without breaking: - `addStep(EntryProcessingStep)` — appends a new step to the pipeline - `removeStep(String stepName)` — removes a step by name - Each step is self-contained; adding or removing one has zero impact on others

Quality attribute improvement: - *Extensibility* — New steps are added by implementing `EntryProcessingStep` and registering in the factory. Zero changes to existing steps or the pipeline orchestrator. - *Maintainability* — Each step is a separate class with a single responsibility. - *Fault tolerance* — Each step is wrapped in try-catch; one step’s failure doesn’t break the pipeline.

Trade-off: Steps execute sequentially (no parallelism). Order matters — e.g., the profanity filter should run before the auto-tag generator so that profane words don’t become tags.

4. Factory Pattern Where: `EntryProcessingPipelineFactory`

Rationale: The factory reads configuration from `roller.properties` and conditionally constructs a pipeline with only the enabled steps. This centralizes creation logic and keeps `EntryEdit` clean — it only calls `EntryProcessingPipelineFactory.createPipeline()` without knowing which steps are active.

Quality attribute improvement: - *Configurability* — Steps are toggled via properties without code changes. - *Single Responsibility* — Pipeline construction logic is isolated from both the pipeline execution and the Struts action.

Trade-off: Adding a new step requires modifying the factory (to add the conditional creation block). This could be further improved with reflection/service loading but would add unnecessary complexity.

5. Strategy Pattern (implicit) Where: Each `EntryProcessingStep` implementation encapsulates a different processing algorithm behind the same interface.

Rationale: All five step implementations implement `EntryProcessingStep` but use completely different algorithms: - Profanity filter: regex-based word boundary matching - Content summarizer: Gemini AI with extractive fallback - Sentiment analysis: keyword-based positive/negative scoring - Reading time estimator: word count / WPM calculation - Auto tag generator: TF-based keyword extraction with stop word filtering

Quality attribute improvement: - *Interchangeability* — Any step can be swapped for a different implementation (e.g., an LLM-based profanity detector) without changing the pipeline. - *Open/Closed Principle* — The pipeline is open for extension (new steps) but closed for modification.

Comparison with Existing Plugin System

Roller already has a `WeblogEntryPlugin` interface with plugins like `ConvertLineBreaksPlugin`, `SmileysPlugin`, etc. Our pipeline is **intentionally separate** from this system:

Aspect	Existing Plugins	New Pipeline
When	Render time (page view)	Save time (entry creation/edit)
Scope	Text rendering only	Title, text, and summary
Control	User-side (per-weblog)	Admin-side (global config)
Persistence	Transforms are not saved	Transforms are persisted
Error handling	Plugin failure breaks rendering	Step failure is isolated

This design follows the hint in the assignment: *“the controls mentioned in this feature are admin-side, NOT user-side”*.

Changes to Existing Code

File	Change
<code>EntryEdit.java</code>	Added 3 lines in <code>save()</code> : create pipeline from factory and execute on entry (line 214-218)
<code>roller.properties</code>	Added 8 pipeline configuration properties (5 step enables + 3 config values)

New Files Created

File	Purpose
<code>business/pipeline/EntryProcessingStep.java</code>	Step 1: <code>getName()</code> , <code>getDescription()</code> , <code>process(WeblogEntry)</code>
<code>business/pipeline/EntryProcessingPipeline.java</code>	Orchestrator: manages step list, executes sequentially with error isolation
<code>business/pipeline/ProfanityFilterStep.java</code>	Step 2: Replaces profane words with asterisks using precompiled regex patterns
<code>business/pipeline/ContentSummarizerStep.java</code>	Step 2: AI-powered summary via Gemini API with extractive fallback; stores in <code>entry.summary</code>
<code>business/pipeline/SentimentAnalysisStep.java</code>	Step 3: Keyword-based sentiment detection; prepends colored badge to entry text
<code>business/pipeline/ReadingTimeEstimatorStep.java</code>	Step 3: Calculates reading time (238 WPM); prepends “X min read” badge
<code>business/pipeline/AutoTagGeneratorStep.java</code>	Step 5: Extracts top-N keywords by frequency, appends as tags to body
<code>business/pipeline/EntryProcessingPipelineFactory.java</code>	Spells out enabled steps from <code>roller.properties</code>

Key Implementation Details

Profanity Filter

- Uses precompiled `Pattern` objects with `\b` (word boundary) matching and `CASE_INSENSITIVE` flag
- Replaces matched words with asterisks of equal length (e.g., “damn” -> “****”)
- Processes title, text, and summary fields independently
- Whole-word matching prevents false positives (e.g., “hell” in “shell” is NOT filtered)

Content Summarizer (AI-powered)

- Calls Gemini API (reuses `pulse.llm.apiKey` and `pulse.llm.apiUrl` from Task 6 config)
- Generates a 2-3 sentence AI summary stored in `entry.setSummary()`
- **Original text is never modified** — preserves full content for permalink views
- Falls back to extractive summarization (first 2-3 sentences) when API is unavailable
- Leverages Roller’s built-in `WeblogEntryPresenter.displayContent(readMoreLink):`
 - **List pages:** shows summary + “Read More” link automatically
 - **Permalink pages:** shows full text automatically
- Skips entries with `<= 2` sentences (too short for a separate summary)

Sentiment Analysis

- Keyword-based approach (no API call) for speed and reliability
- 40 positive words (great, excellent, amazing, love, ...) and 40 negative words (terrible, awful, hate, ...)
- Score = `positiveCount - negativeCount`; threshold ± 1 for classification
- Prepends a colored badge: green (Positive), gray (Neutral), red (Negative)
- Also stores sentiment in `entry.searchDescription` for SEO meta tags

Reading Time Estimator

- Calculates reading time based on word count / 238 WPM (configurable)
- Prepends a blue “X min read” badge to entry text
- Minimum 1 minute for short entries

Auto Tag Generator

- Strips HTML, tokenizes text, filters stop words and short words (< 4 chars)
- Counts word frequency and picks top-N most frequent words
- Appends tags as `<p class="auto-tags"><em>Tags: #keyword1 #keyword2 ...</em></p>`
- Configurable max tags (default 5, set via `pipeline.step.autoTagGenerator.maxTags`)

Configuration

In `roller.properties`:

```
# Execution order: Profanity -> Summarizer -> Sentiment -> ReadingTime -> AutoTag

# Step 1: Profanity Filter
pipeline.step.profanityFilter.enabled=true

# Step 2: Content Summarizer (AI-powered, uses pulse.llm.apiKey)
pipeline.step.contentSummarizer.enabled=true

# Step 3: Sentiment Analysis (keyword-based)
pipeline.step.sentimentAnalysis.enabled=true

# Step 4: Reading Time Estimator
pipeline.step.readingTimeEstimator.enabled=true
pipeline.step.readingTimeEstimator.wordsPerMinute=238

# Step 5: Auto Tag Generator
pipeline.step.autoTagGenerator.enabled=true
pipeline.step.autoTagGenerator.maxTags=5
```

To disable any step, set its `enabled` property to `false`. No code changes or recompilation needed — just restart the application.

Pipeline execution order rationale: 1. Profanity filter runs first — cleans text before downstream steps analyze it 2. Content summarizer runs on clean text — generates AI summary before badges are prepended 3. Sentiment analysis runs on clean text — accurate keyword matching 4. Reading time estimator runs before tags — tag section shouldn’t count as reading content 5. Auto tag generator runs last — tags are appended and don’t affect other steps

How to Add a New Step

1. Create a class implementing `EntryProcessingStep`
2. Add a conditional block in `EntryProcessingPipelineFactory.createPipeline()`
3. Add an `enabled` property in `roller.properties`

No existing steps or the pipeline orchestrator need to be modified.

Task 3: Bug Reporting System

1. Feature Description

The Bug Report module extends Apache Roller with a complete workflow for submitting, tracking, and managing software issue reports within the application.

Regular users can create, view, edit, and delete their own bug reports from the authoring interface. Administrators can review all reports from a centralized dashboard, filter them by status, add administrative notes, change report status, and delete reports when required.

The feature is designed to integrate with existing Roller architecture rather than introducing a separate subsystem. It reuses Struts 2 UI flow, JPA-based manager patterns, configuration systems, and existing security mechanisms.

In addition to CRUD functionality, the module includes an extensible notification pipeline that allows the system to notify administrators and reporters via email, with support for future extensions such as Slack or Microsoft Teams.

2. Implementation

Backend The backend is structured into four layers:

Domain Model The `BugReport` entity stores report details such as title, description, severity, type, owner, timestamps, status, page URL, reproduction steps, expected/actual behavior, and admin notes.

Data Access Layer The `BugReportManager` interface and `JPABugReportManagerImpl` handle persistence using Roller's manager-based architecture.

Workflow Layer The `BugReportWorkflowService` manages validation, command execution, event creation, and notification triggering.

Notification Layer The `BugReportNotificationService`, `BugReportEventPublisher`, `BugNotificationChannel`, and `BugNotificationMessageStrategy` handle modular and extensible notification dispatch.

Backend Functionality

- `create()` validates input, stores the report, and notifies admins
- `update()` validates ownership and updates report
- `changeStatus()` validates transitions and notifies both admin and reporter
- `delete()` removes the report and triggers notifications

Admin recipient resolution follows:

```
admin users -> site.adminemail -> mail.username
```

Notification delivery is currently email-based using `EmailBugNotificationChannel`, but is abstracted through `BugNotificationChannel` for extensibility.

Status Lifecycle

- OPEN -> TRIAGED
 - TRIAGED -> RESOLVED
 - RESOLVED -> TRIAGED (reopen)
-

Frontend The frontend is implemented using Struts 2 actions and JSP views integrated into Roller UI.

User Features:

- View bug reports
- Create/edit reports
- Delete owned reports

Admin Features:

- Central dashboard
- Filter by status
- Update status with notes
- Delete reports

Screenshots

- User bug report list page
 - Bug creation/edit form
 - Admin dashboard
 - Status change interface
 - Creation mail to admin
-

3. Assumptions / Constraints

- Users must have `EDIT_DRAFT` permission to access bug reporting
- Admin dashboard requires `ADMIN` permission
- Users can modify only their own reports
- Reporter notifications are sent only on status change or deletion
- Admin notifications are sent for all actions
- Email delivery depends on valid SMTP configuration
- Only one notification channel (`EmailBugNotificationChannel`) is currently implemented
- Status model is limited to OPEN, TRIAGED, and RESOLVED
- User and admin interfaces are separate

4. Files Modified / Added

Domain and Persistence

- `BugReport.java`
 - `BugReport.orm.xml`
 - `createdb.vm`
 - `persistence.xml`
-

Business and Workflow

- `BugReportManager.java`
 - `JPABugReportManagerImpl.java`
 - `BugReportCommand.java`
 - `CreateBugReportCommand.java`
 - `UpdateBugReportCommand.java`
 - `DeleteBugReportCommand.java`
 - `ChangeStatusBugReportCommand.java`
 - `BugReportWorkflowService.java`
 - `BugReportServiceFactory.java`
-

Notifications

- `BugReportEvent.java`
 - `BugReportEventType.java`
 - `BugReportAudience.java`
 - `BugReportEventPublisher.java`
 - `BugReportNotificationService.java`
 - `BugNotificationChannel.java`
 - `EmailBugNotificationChannel.java`
 - `BugNotificationChannelFactory.java`
 - `BugNotificationMessage.java`
 - `BugNotificationMessageStrategy.java`
 - `AdminBugNotificationMessageStrategy.java`
 - `ReporterStatusNotificationMessageStrategy.java`
-

UI and Views

- `BugReportBean.java`
- `BugReports.java`
- `BugReportsAdmin.java`
- `BugReports.jsp`
- `BugReportEdit.jsp`

- BugReportsAdmin.jsp
-

Framework and Configuration Integration

- struts.xml
 - tiles.xml
 - editor-menu.xml
 - admin-menu.xml
 - ApplicationResources.properties
 - runtimeConfigDefs.xml
 - Weblogger.java
 - WebloggerImpl.java
 - JPAWebloggerImpl.java
 - JPAWebloggerModule.java
-

Tests and Documentation

- BugReportTest.java
 - BugReportCRUDTest.java
 - BugReportBeanTest.java
 - BugReportWorkflowServiceNotificationTest.java
 - EmailBugNotificationSmtSmokeTest.java
 - project2_bug-report-task.md
 - bug-report-module-class-diagram.puml
-

5. Design Patterns Used

- **Repository Pattern** The **Repository Pattern** is implemented through `BugReportManager` and its concrete implementation `JPABugReportManagerImpl`. This pattern abstracts persistence logic from the rest of the application and aligns with Roller's existing manager-based architecture. It ensures that database operations are centralized and isolated from business logic, improving maintainability and consistency across the system.
- **Command Pattern** The **Command Pattern** is used extensively in the workflow layer through `BugReportCommand` and its concrete implementations such as `CreateBugReportCommand`, `UpdateBugReportCommand`, `DeleteBugReportCommand`, and `ChangeStatusBugReportCommand`. Each operation on a bug report is encapsulated as a separate command, allowing validation, execution, and testing to be handled independently. This improves modularity and makes the system easier to extend with additional operations in the future.
- **Strategy Pattern** The **Strategy Pattern** is applied in the notification subsystem using `BugNotificationMessageStrategy` and its implementations `AdminBugNotificationMessageStrategy` and `ReporterStatusNotificationMessageStrategy`. Since different audiences require different message formats, this pattern enables flexible message generation without modifying

the notification pipeline. It improves extensibility and allows new message formats to be introduced easily.

- **Observer (Event) Pattern** An **Observer (Event-Driven) Pattern** is realized through `BugReportEvent` and `BugReportEventPublisher`. The workflow service generates events when actions such as create, update, delete, or status change occur, and these events are propagated to the notification system. This decouples the core business logic from notification handling, ensuring that additional reactions to events can be introduced without modifying existing workflows.
 - **Factory Pattern** The **Factory Pattern** is used in `BugNotificationChannelFactory` and `BugReportServiceFactory` to centralize object creation. This prevents direct instantiation of concrete classes and allows runtime configuration of notification channels. As a result, the system can support additional channels such as Slack or Microsoft Teams with minimal changes.
 - **Template Method Pattern** **Template Method Pattern** is leveraged through the existing `UIAction` class provided by Apache Roller. All UI actions inherit common behavior such as security checks, session handling, and request processing, ensuring consistency across user and admin interfaces while reducing code duplication.
-

6. Rationale Behind Pattern Selection

- The Repository pattern aligns with existing Roller architecture and isolates database logic
 - The Command pattern encapsulates different operations and improves extensibility
 - The Strategy pattern enables flexible notification message formatting
 - The Observer pattern decouples workflow from notification delivery
 - The Factory pattern centralizes object creation and supports future extensibility
 - The Template Method pattern ensures consistency with Roller UI behavior
-

7. User Flow

Normal User Flow

1. User logs into Roller and opens a weblog
 2. User navigates to the bug report section
 3. User creates a new bug report
 4. The system validates and stores the report using `BugReportWorkflowService`
 5. A `BugReportEvent` is generated and admins are notified
 6. User can edit or delete their report
-

Admin Flow

1. Admin logs into Roller and opens the dashboard
2. Admin views or filters reports
3. Admin updates status using `ChangeStatusBugReportCommand`

4. The system persists changes and triggers notifications
 5. Admin can delete reports
-

Notification Flow

1. A workflow action is executed via `BugReportWorkflowService`
 2. A `BugReportEvent` is generated
 3. `BugReportNotificationService` determines recipients
 4. A `BugNotificationMessageStrategy` generates the message
 5. `BugReportEventPublisher` sends notifications via `EmailBugNotificationChannel`
-

8. UML Diagram

(See *UML Diagrams* section below)

Task 4: Admin Dashboard

Overview

A Site Summary Dashboard accessible to admin users that provides a high-level overview of key site metrics. Supports two view modes — **Minimalist** (3 metrics) and **Full** (8 metrics) — toggled via buttons. Uses the Builder pattern to separate view definition (which metrics to include) from data-fetching logic (how each metric computes its value).

Requirements Addressed

- Admin-only dashboard showing at least 5 site-wide metrics (Full view shows 9)
- Two distinct view modes: Minimalist and Full
- View definition is decoupled from data-fetching logic (Builder pattern)
- Each metric computes independently; one failure does not break others
- Accessible via the admin navigation menu

Design Patterns Used

10. Builder Pattern **Where:** `DashboardReportBuilder` (builder), `DashboardReport` (product), `DashboardMetric` (component interface)

Rationale: The assignment states “*definition of each view shouldn’t be described with the logic to fetch data.*” The Builder pattern directly addresses this: the `DashboardReportBuilder.buildMinimalistReport()` and `buildFullReport()` static methods define **which** metrics each view includes (view definition), while each `DashboardMetric` implementation encapsulates **how** its data is fetched (data-fetching logic). These two concerns are in completely separate classes.

How it works:

```
// VIEW DEFINITION --- which metrics to include (in DashboardReportBuilder)
public static DashboardReport buildMinimalistReport() {
    return new DashboardReportBuilder()
        .setViewName("Minimalist")
        .addMetric(new TotalUsersMetric())
        .addMetric(new TotalWeblogsMetric())
        .addMetric(new TopCategoryMetric())
        .build();
}

// DATA-FETCHING LOGIC --- how each metric computes its value (in TotalUsersMetric)
public MetricResult compute() {
    UserManager umgr = WebloggerFactory.getWeblogger().getUserManager();
    long count = umgr.getUserCount();
    return new MetricResult(getName(), getLabel(), String.valueOf(count));
}
```

Quality attribute improvement: - *Extensibility* — New metrics are added by implementing `DashboardMetric`; new views by calling different `addMetric()` combinations. Neither change affects the other. - *Fault tolerance* — `build()` wraps each `metric.compute()` in try-catch; a failing metric returns an “Error” result without blocking others. - *Maintainability* — Each metric is a separate, testable class with a single responsibility.

Trade-off: Each metric makes its own database call. For 8 metrics, this means 8 queries per page load. Acceptable for an admin-only page, but could be optimized with caching if needed.

11. Strategy Pattern (Metrics) **Where:** `DashboardMetric` (interface) with 8 implementations

Rationale: Each metric implementation encapsulates a different data-fetching algorithm (user count query, trending query, aggregate comment query, etc.) behind the same `DashboardMetric` interface. The builder treats all metrics uniformly — it doesn’t know or care how each one fetches its data.

Quality attribute improvement: - *Open/Closed Principle* — New metrics (e.g., “Average Posts Per User”, “Most Active Day of Week”) can be added without modifying existing code. - *Interchangeability* — Any metric can be swapped for a different implementation (e.g., a cached version).

Metrics

#	Metric	Data Source	What It Shows
1	Total Users	<code>UserManager.getUserCount()</code>	Total registered user count
2	Total Weblogs	<code>WeblogManager.getWeblogCount()</code>	Total weblog count
3	Total Entries	<code>WeblogEntryManager.getEntryCount()</code>	Total weblog entries across all weblogs
4	Total Comments	<code>WeblogEntryManager.getCommentCount()</code>	Total comments across all entries
5	Top Category	<code>WeblogCategory.retrieveWeblogEntries()</code>	Top weblog entries with the most blog entries across all weblogs

#	Metric	Data Source	What It Shows
6	Most Commented Category	WeblogCategory + WeblogEntry.getCommentCount()	Category with the most comments across all weblogs
7	Most Starred Blog	StarManager.getTrendingWeblogs(6)	Weblog with the most stars
8	Top Active Users	UserManager + WeblogManager	Top 3 users by weblog count
9	Most Commented Weblog	WeblogManager.getMostCommentedWeblogs(1)	Weblog with highest comment count

View composition: - Minimalist View (3 metrics): Total Users, Total Weblogs, Top Category
- Full View (9 metrics): All of the above

Changes to Existing Code

File	Change
struts.xml	Added siteSummary action mapping in weblogger-admin package
tiles.xml	Added .SiteSummary tile definition
admin-menu.xml	Added siteSummary menu item with tabbedmenu.admin.siteSummary label
ApplicationResources.properties	Added 4 i18n keys: siteSummary.title, siteSummary.subtitle, siteSummary.prompt, tabbedmenu.admin.siteSummary

New Files Created

File	Purpose
Interface & Data Classes	
business/dashboard/DashboardMetricInterface.java	Interface: getName(), getLabel(), compute() -> MetricResult
business/dashboard/MetricResult.java	Mutable result: name, label, value, optional details list
business/dashboard/DashboardReport.java	Assembled report: viewName + unmodifiable list of MetricResult
Builder	
business/dashboard/DashboardReportBuilder.java	Builder with addMetric(), build(), and static buildMinimalistReport() / buildFullReport()
Metric Implementations (9)	
business/dashboard/TotalUsersMetric.java	Fetches user count from UserManager
business/dashboard/TotalWeblogsMetric.java	Fetches weblog count from WeblogManager
business/dashboard/TotalEntriesMetric.java	Fetches entry count from WeblogEntryManager
business/dashboard/TotalCommentsMetric.java	Fetches comment count from WeblogEntryManager

File	Purpose
business/dashboard/TopCategoryMetricFactory.java	Finds a category with most entries across all weblogs
business/dashboard/MostCommentedCategoryMetric.java	Finds a category with most comments across all weblogs
business/dashboard/MostStarredBlogMetric.java	Finds a weblog with most stars via <code>StarManager</code>
business/dashboard/TopActiveUsersMetric.java	Finds users by weblog count (top 3)
business/dashboard/MostCommentedWeblogMetric.java	Finds a weblog with most comments
Struts Action	
ui/struts2/admin/SiteSummary.java	Admin action with <code>view</code> parameter; delegates to builder
JSP	
WEB-INF/jsp/admin/SiteSummary.jsp	Bootstrap panel cards with view toggle buttons

Key Implementation Details

- **View toggle:** The `view` parameter (“full” or “minimalist”) is passed as a URL parameter. The action delegates to the appropriate builder method. Active view is highlighted with `btn-primary`.
- **Error isolation:** If any metric’s `compute()` throws, the builder catches the exception, logs it, and inserts an “Error” result — other metrics still display normally.
- **Admin-only access:** `SiteSummary.requiredGlobalPermissionActions()` returns `GlobalPermission.ADMIN`; non-admins cannot access the page.
- **Consistent with Roller patterns:** Follows the same action -> tiles -> JSP pattern as `CacheInfo`, `GlobalConfig`, and other existing admin pages.
- **Responsive layout:** Cards are displayed in a 3-column Bootstrap grid that wraps on smaller screens.

How to Use

1. Log in to Apache Roller as an **admin** user
2. Click “**Site Summary**” in the admin navigation tabs
3. The Full view is shown by default with all 8 metrics
4. Click “**Minimalist View**” to see only 3 key metrics
5. Click “**Full View**” to return to the comprehensive view

Task 5: Web Translation and Caching

1. Feature Description

Task 5 introduces a translation subsystem for public Roller weblog pages and combines it with section-level caching to avoid repeated calls to external translation providers.

The feature allows users to open a public weblog page, select a source language, target language, and translation provider from a floating widget, and translate the visible text content without altering the page structure.

The implementation supports translation across multiple public views including weblog pages, permalinks, search pages, tags, archives, and frontpage views.

Key capabilities include:

- Translation of rendered weblog text on public pages
 - Runtime switching between `MyMemoryTranslationProvider` and `SarvamTranslationProvider`
 - Support for multiple languages including Marathi
 - Restore-to-original functionality
 - Automatic reuse of previously selected translation settings
 - Section-level caching to avoid redundant translations
-

2. Implementation

Backend The backend is built around a translation pipeline exposed through `TranslationServlet`.

The `TranslationServlet` receives JSON requests from the frontend, validates language inputs, selects the appropriate provider using `TranslationProviderFactory`, and delegates translation to `TranslationCacheService`.

Core Components **Translation Entry Point** The `TranslationServlet` handles request parsing, validation, provider resolution, and response generation.

Caching Layer The `TranslationCacheService` performs section-level caching. It computes a SHA-256 hash over normalized text content (whitespace-trimmed and normalized) to detect meaningful changes.

Cache keys include:

- provider
- source language
- target language
- content hash

This ensures cached translations are reused only when the content is unchanged.

Language Normalization The `TranslationLanguageSupport` class standardizes language codes (e.g., `en-US`, `mr-IN`) and maps them to provider-specific formats such as `mr-IN`.

Provider Layer The `TranslationProvider` interface defines a common contract for translation providers.

Concrete implementations include:

- `MyMemoryTranslationProvider`
- `SarvamTranslationProvider`

The `SarvamTranslationProvider` reads API keys from Roller configuration, while both providers support batch translation while preserving input order.

Backend Functionality

- `TranslationServlet` supports section-based translation requests
- `TranslationProviderFactory` selects providers dynamically

- `TranslationCacheService` reuses cached sections and retranslates only changed content
 - `TranslationLanguageSupport` ensures consistent language handling
 - Providers perform batch translation and preserve response order
-

Frontend The frontend is implemented as a shared JavaScript widget using:

- `translation.js`
- `translation.css`

The widget is injected on page load and performs the following steps:

- Detects source language from page attributes
- Extracts visible text nodes and groups them into sections
- Sends sections to `/roller-services/translate`
- Replaces translated text in the DOM without altering layout

Additional features include:

- Local storage of translation preferences
 - Restore-to-original functionality
 - Automatic reapplication of translation on revisit
-

Screenshots

1. Webpage with translation widget
 2. Translated page (Marathi)
 3. Cached translation reuse indicator
 4. Provider switch (`MyMemoryTranslationProvider` vs `SarvamTranslationProvider`)
-

3. Assumptions / Constraints

- Translation is available only on theme views that include the widget
 - Supported languages: `en`, `hi`, `bn`, `ta`, `te`, `kn`, `mr`
 - Source language detection uses page `lang` attribute first
 - Only visible text nodes are translated (no images, links, attributes)
 - Cache is in-memory within `TranslationCacheService`
 - Cache is not persistent across server restarts
 - Cache reuse depends on normalized content and translation parameters
 - `SarvamTranslationProvider` requires a valid API key
 - Frontend integration is limited to selected theme views
-

4. Files Modified / Added

Backend

- TranslationServlet.java
 - TranslationCacheService.java
 - TranslationLanguageSupport.java
 - TranslationProvider.java
 - TranslationProviderFactory.java
 - MyMemoryTranslationProvider.java
 - SarvamTranslationProvider.java
 - TranslationSectionRequest.java
 - TranslationSectionResponse.java
 - web.xml
 - roller.properties
-

Frontend

- translation.js
 - translation.css
-

Theme Integration

- weblog.vm
 - permalink.vm
 - searchresults.vm
 - archives.vm
 - tags_index.vm
 - _header.vm, _footer.vm
-

Tests and Documentation

- TranslationCacheServiceTest.java
 - TranslationLanguageSupportTest.java
 - task5-translation-system.md
 - task5-translation-system-class-diagram.puml
-

5. Design Patterns Used

- **Strategy Pattern** The **Strategy Pattern** is used through the **TranslationProvider** interface and its implementations, **MyMemoryTranslationProvider** and **SarvamTranslationProvider**. This allows the system to support multiple translation providers with different APIs and configurations while exposing a uniform interface to the rest of the application. As a result,

the translation logic remains independent of provider-specific details, improving modularity and enabling easy extension to additional providers in the future.

- **Factory Pattern** The **Factory Pattern** is implemented via `TranslationProviderFactory`, which centralizes the creation of translation provider instances. This ensures that the servlet layer depends only on abstractions rather than concrete implementations, thereby reducing coupling and improving maintainability. It also simplifies the integration of new providers by localizing object creation logic.
- **Cache Pattern (Decorator-like behavior)** The Cache Pattern is realized through `TranslationCacheService`, which stores translations at a section level to avoid redundant API calls. By computing a hash of normalized content, the system ensures that translations are reused only when the content remains unchanged. This significantly improves performance, reduces external API usage, and enhances responsiveness for repeated page visits, while introducing additional complexity in cache management and invalidation.
- **Adapter-like Pattern** A **Template Method-style workflow** is followed in `TranslationServlet`, which defines a fixed sequence of steps for handling translation requests, including request parsing, language normalization, provider selection, cache lookup, and response generation. While not implemented through classical inheritance, this structured pipeline ensures consistency and improves readability and maintainability of the request-handling process.
- **Template Method Style Flow Adapter-like pattern** is employed in `TranslationLanguageSupport` to normalize and map language codes across different system components. Since the frontend, internal system, and external providers use varying language formats, this layer ensures compatibility and centralizes language handling logic, improving maintainability and reducing errors.

6. Rationale Behind Pattern Selection

- Strategy allows switching between translation providers without modifying core logic
- Factory centralizes provider selection and simplifies extensibility
- Cache reduces redundant API calls and improves performance
- Normalization layer ensures compatibility between frontend and provider formats
- Template-style flow keeps request processing structured and maintainable

7. User Flow

1. User opens a public weblog page with translation widget
2. `translation.js` initializes and restores saved preferences
3. User selects target language and provider
4. Text content is extracted and grouped into sections
5. Sections are sent to `/roller-services/translate`
6. `TranslationServlet` processes the request and invokes `TranslationCacheService`
7. Cached sections are reused, and only new content is translated
8. Translated content is returned to frontend
9. DOM text is replaced without altering layout

10. Settings are saved for future visits
 11. If content changes, only modified sections are retranslated
-

8. UML Diagram

(See *UML Diagrams* section below)

Task 6: Community Pulse Dashboard

Overview

A dashboard that helps weblog authors understand what their readers are talking about by summarizing and organizing comment discussions. Combines classical lightweight indicators (6A) with intelligent conversation breakdown using multiple methods (6B).

Requirements Addressed

- **6A Discussion Overview:** 5 lightweight indicators computed using classical methods (no LLM)
- **6B Conversation Breakdown:** Organized breakdown with themes, representative comments per theme, and overall recap
- **6B Methods:** Two distinct methods — TF-IDF (classical) and Hybrid (local clustering + LLM labeling)
- **6B Dynamic Selection:** Strategy is automatically selected based on comment count and LLM availability
- **6B Representative Comments:** Each theme includes the top 2 most relevant comments
- Design allows easily switching between insight-generation methods

Design Patterns Used

6. Strategy Pattern Where: `BreakdownStrategy` (interface), `TfIdfBreakdownStrategy`, `HybridLlmBreakdownStrategy`

Rationale: The assignment requires “at least two distinct methods” that the system can “easily switch between... depending on available resources.” The Strategy pattern encapsulates each breakdown algorithm behind a common `BreakdownStrategy` interface, allowing the system to swap methods at runtime without any code changes.

How it works: - `TfIdfBreakdownStrategy` — fully classical TF-IDF keyword clustering with no external dependencies - `HybridLlmBreakdownStrategy` — uses TF-IDF locally for clustering, then sends a compact prompt to Gemini AI for polished labels and recap (minimizes API cost) - Both implement `analyze(CommentData)` and return a `ConversationBreakdown` with themes + representative comments + recap

Quality attribute improvement: - *Extensibility* — New breakdown methods (e.g., a pure LLM strategy, or an embedding-based one) can be added by implementing `BreakdownStrategy`. Zero changes to existing code. - *Configurability* — The user can switch methods from the UI dashboard at runtime. - *Cost efficiency* — LLM is used selectively, not for every request.

Trade-off: Each strategy maintains its own duplicate stop-word list. This could be extracted into a shared utility, but keeping strategies self-contained simplifies testing and reasoning.

7. Factory Pattern (Dynamic Strategy Selection) Where: BreakdownStrategySelector

Rationale: The selector acts as a factory that dynamically chooses the right strategy based on two factors: (1) the number of comments (small sets don't benefit from LLM overhead) and (2) whether an LLM API key is configured.

Selection logic: | Comment Count | LLM Available | Strategy Selected | |
 |-----| | 0-5 | Any | TF-IDF (too few for meaningful clustering) | | 6-30 | No | TF-IDF |
 | 6-30 | Yes | Hybrid LLM | | 31+ | No | TF-IDF | | 31+ | Yes | Hybrid LLM (most benefit from polished labels) |

Quality attribute improvement: - *Sustainability* — Avoids wasting API calls on trivial comment sets - *Fault tolerance* — Falls back to TF-IDF if LLM call fails

Trade-off: Thresholds are hardcoded. Could be made configurable via `roller.properties`, but current values are reasonable defaults.

8. Facade Pattern Where: CommunityPulseAnalyzer

Rationale: The facade provides a single entry point that coordinates two independent subsystems (6A indicators and 6B breakdown). The Struts action only needs to call `analyzer.analyze(entryId)` — it doesn't know about individual indicators, strategies, or comment fetching.

Quality attribute improvement: - *Simplicity* — Complex multi-step analysis reduced to a single method call - *Maintainability* — Internal structure can change without affecting the action layer

9. Template Method Pattern (implicit) Where: DiscussionIndicator (interface) with 5 implementations

Rationale: All indicators follow the same contract: `compute(CommentData) -> Map<String, Object>`. The `DiscussionOverview` iterates over all registered indicators and calls `compute()` on each. Each implementation decides its own computation logic.

Quality attribute improvement: - *Open/Closed Principle* — New indicators are added by creating a class that implements `DiscussionIndicator` and registering it in `DiscussionOverview`. No existing indicators are modified. - *Error isolation* — Each indicator's `compute()` is wrapped in try-catch; one failure doesn't break others.

6A: Discussion Overview Indicators

All 5 indicators are classical and computationally inexpensive (no LLM):

#	Indicator	What It Computes
1	Activity Level	Classifies as Silent/Cold/Warm/Hot/On Fire based on count + comments-per-day rate

#	Indicator	What It Computes
2	Response Type Breakdown	% of comments classified as questions, positive feedback, debate, or general (regex/keyword matching)
3	Recurring Keywords	Top 5 most frequent meaningful words (word frequency with stop-word filtering, HTML stripping)
4	Top Contributors	Top 3 most active commenters by comment count
5	Unique Commenter Count	Distinct names + diversity ratio (unique/total) -> Highly Diverse / Moderate / Low / Dominated by Few

6B: Conversation Breakdown Methods

Method 1: TF-IDF Keyword Clustering (Classical) Fully local, no external dependencies:

1. **Tokenize** each comment (strip HTML, remove stop words)
2. **Compute TF-IDF** scores per comment
3. **Extract global top keywords** by cumulative TF-IDF score
4. **Cluster** comments by their highest-scoring keyword
5. **Label** each cluster from its primary keyword
6. **Pick representative comments** — top 2 comments per cluster ranked by TF-IDF score for the cluster keyword
7. **Generate recap** by summarizing cluster sizes and labels

Method 2: Hybrid (Local Clustering + LLM Labeling) Minimizes LLM usage by doing heavy lifting locally:

1. **Run TF-IDF clustering** locally (same as Method 1) — **free**
2. **Build compact prompt** with only cluster keywords + 2 representative comments per cluster
3. **Single LLM call** to Gemini AI for polished human-readable theme labels and an overall recap
4. **Fallback** to TF-IDF labels if LLM call fails (network error, quota exceeded, etc.)

Why this is sustainable: Only one API call per analysis, with a small payload (~500 tokens). The local clustering eliminates the need to send all comments to the LLM.

Changes to Existing Code

File	Change
struts.xml	Added <code>communityPulse</code> action mapping
tiles.xml	Added <code>.CommunityPulse</code> tile definition
editor-menu.xml	Added “Community Pulse” menu item in editor tab
ApplicationResources.properties	Added 16 i18n keys for Community Pulse UI
roller.properties	Added 2 LLM configuration properties (<code>pulse.llm.apiKey</code> , <code>pulse.llm.apiUrl</code>)

New Files Created

File	Purpose
Interfaces	
business/pulse/DiscussionIndicator.java	Interface for lightweight indicators: compute(CommentData) -> Map
business/pulse/BreakdownStrategy.java	Interface for breakdown methods: analyze(CommentData) -> ConversationBreakdown
Data classes	
business/pulse/CommentData.java	Immutable snapshot of comments for a single entry
business/pulse/ConversationTheme.java	A theme with label, keywords, representative comments, count
business/pulse/ConversationBreakdown.java	Breakdown result: themes + recap + method used
business/pulse/PulseResult.java	Full analysis result combining indicators + breakdown
6A Indicators (5)	
business/pulse/ActivityLevelIndicator.java	Cold/Warm/Hot/On Fire classification
business/pulse/ResponseTypeIndicator.java	Question/Positive/Debate/General breakdown
business/pulse/RecurringKeywordsIndicator.java	Top 5 keywords by frequency
business/pulse/TopContributorsIndicator.java	Top 3 commenters by count
business/pulse/UniqueCommenterIndicator.java	Unique commenters + diversity ratio
6A Aggregator	
business/pulse/DiscussionOverview.java	Runs all 5 indicators, collects results
6B Strategies (2)	
business/pulse/TfIdfBreakdownStrategy.java	Classic TF-IDF clustering with local recap
business/pulse/HybridLlmBreakdownStrategy.java	Local TF-IDF + one LLM call for polish
6B/6C Selector	
business/pulse/BreakdownStrategySelector.java	Dynamic factory choosing strategy by comment count + config
Facade	
business/pulse/CommunityPulseAnalyzer.java	Single entry point coordinating indicators + breakdown
UI	
ui/struts2/editor/CommunityPulse.java	Struts action with entry selector + analysis
WEB-INF/jsp/editor/CommunityPulseDisplay.jsp	Dashboard JSP with indicator cards + theme panels

Configuration

In roller.properties:

```
# Leave empty for TF-IDF only (no LLM calls)
pulse.llm.apiKey=
pulse.llm.apiUrl=https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-flash:generateText
```

To enable the Hybrid method, set `pulse.llm.apiKey` to a valid Gemini API key. The system automatically selects the best method based on comment count and API availability.

How to Use

1. Log in to Apache Roller and navigate to your weblog

2. Click “**Community Pulse**” in the editor menu tab
3. Select an entry from the dropdown (shows entries with comments)
4. Click “**Analyze**”
5. View the 5 discussion indicators (activity level, response types, keywords, contributors, commenter diversity)
6. View the conversation breakdown with themes, representative comments, and recap
7. If LLM is configured, use the “**Switch Method**” buttons to compare TF-IDF vs Hybrid results

UML Diagrams

Before — Existing comment infrastructure with no analytics:

After — Full Community Pulse system with indicators, strategies, and dashboard:

Design Patterns Summary

#	Pattern	Location	Task	Justification
1	DAO	StarManager / JPASStarManagerImpl	1A/1B	Separates persistence from business logic; consistent with Roller’s architecture
2	Dependency Injection	Guice bindings for StarManager	1A/1B	Loose coupling; interface-based dependency; singleton lifecycle management
3	Chain of Responsibility	EntryProcessingStep2 / EntryProcessingPipeline		Sequential processing with independent, add/removable steps
4	Factory	EntryProcessingPipelineFactory		Centralizes pipeline construction; reads config to conditionally enable steps
5	Strategy (Pipeline)	Each EntryProcessingStep implementation	2	Different algorithms behind the same interface; interchangeable steps
6	Strategy (Breakdown)	BreakdownStrategy / TfIdfBreakdownStrategy / HybridLlmBreakdownStrategy	6	Switchable insight-generation methods (classical vs LLM-enhanced)
7	Factory (Dynamic)	BreakdownStrategySelector		Selects strategy at runtime based on comment count and LLM availability
8	Facade	CommunityPulseAnalyzer		Single entry point coordinating 6A indicators and 6B breakdown subsystems

#	Pattern	Location	Task	Justification
9	Template Method	DiscussionIndicator6 with 5 implementations		Common contract for indicators; each computes independently; error isolation
10	Builder	DashboardReportBuilder / DashboardReport / DashboardMetric		Separates view definition from data-fetching logic; flexible view composition
11	Strategy (Metrics)	9 DashboardMetric implementations	4	Different data-fetching algorithms behind uniform interface; extensible

UML Diagrams

Task 1A/1B — User Highlights

Before — No star/favourite mechanism; only `WeblogHitCount` for page hits:

After — Full starring system with `StarManager`, POJOs, actions, pagination, and trending:

Task 2 — Transforming Feeds

Before — Only render-time `WeblogEntryPlugin` system; no save-time processing:

After — New admin-side `EntryProcessingPipeline` with Chain of Responsibility pattern:

Task 4 — Admin Dashboard

Before — Admin pages are isolated management tools with no centralized site summary. Data access methods (`getUserCount`, `getWeblogCount`, etc.) exist but are unused by the admin UI:

After — New `SiteSummary` action uses `DashboardReportBuilder` (Builder pattern) to assemble metric-based reports. Each `DashboardMetric` implementation encapsulates its own data-fetching logic. Two predefined views (Minimalist: 3 metrics, Full: 9 metrics) are composed by calling different combinations of `addMetric()`:

Testing

Task 1 — Unit Tests (10 POJO tests passing + 1 integration test)

Test Class	# Tests	What's Tested	Type
<code>UserWeblogStarTest</code>	5	POJO getters/setters, equals/hashCode, UUID generation	Unit (no DB)
<code>UserEntryStarTest</code>	5	POJO getters/setters, equals/hashCode, UUID generation	Unit (no DB)

Test Class	# Tests	What's Tested	Type
StarManagerTest	—	Full CRUD, duplicate prevention, trending queries, star counts	Integration (requires DB)

Run Task 1 POJO tests:

```
mvn test -pl app -Dtest="org.apache.roller.weblogger.pojos.UserWeblogStarTest,org.apache.roller.weblogger.pojos.WeblogTest,org.apache.roller.pojos.UserTest"
```

Run Task 1 integration test (requires running Derby DB via `mvn jetty:run`):

```
mvn test -pl app -Dtest="org.apache.roller.weblogger.business.StarManagerTest"
```

Note: `StarManagerTest` is an integration test that requires the full Roller DB schema to be initialized. This is a pre-existing infrastructure constraint — all Roller business-layer integration tests (e.g., `WeblogTest`, `UserTest`) require the same DB setup and fail identically without it.

Task 2 — Unit Tests (59 total, all passing)

Test Class	# Tests	What's Tested
ProfanityFilterStepTest	8	Word boundary matching, case insensitivity, title/text/summary filtering, null handling
ContentSummarizerStepTest	11	AI summary fallback, original text preservation, extractive summarization, HTML stripping, short text skipping
SentimentAnalysisStepTest	12	Positive/negative/neutral classification, badge HTML, search description update, mixed sentiment
ReadingTimeEstimatorStepTest	11	Short/medium/long text timing, badge prepend, custom WPM, null/empty handling
AutoTagGeneratorStepTest	9	Tag extraction, stop word exclusion, HTML stripping, max tags limit, null/blank text
EntryProcessingPipelineTest	8	Add/remove steps, execution order, null entry, full 5-step integration, text preservation

Run all Task 2 tests:

```
mvn test -pl app -Dtest="org.apache.roller.weblogger.business.pipeline.ProfanityFilterStepTest,org.apache.roller.weblogger.business.pipeline.ContentSummarizerStepTest,org.apache.roller.weblogger.business.pipeline.SentimentAnalysisStepTest,org.apache.roller.weblogger.business.pipeline.ReadingTimeEstimatorStepTest,org.apache.roller.weblogger.business.pipeline.AutoTagGeneratorStepTest,org.apache.roller.weblogger.business.pipeline.EntryProcessingPipelineTest"
```

Task 4 — Unit Tests (22 total, all passing)

Test Class	# Tests	What's Tested
MetricResultTest	5	Constructors, null details handling, immutability, empty values
DashboardReportTest	4	View name, results list, null results, immutability, order preservation
DashboardReportBuilderTest	8	Empty build, single/multiple metrics, failing metrics, chaining, order preservation
SiteSummaryActionTest	5	Default view, view setter, admin permission, weblog not required, null report before execute

Run all Task 4 tests:

```
mvn test -pl app -Dtest="MetricResultTest,DashboardReportTest,DashboardReportBuilderTest,SiteSummaryActionTest"
```

Task 6 — Unit Tests (48 total, all passing)

Test Class	# Tests	What's Tested
ActivityLevelIndicatorTest	6	Silent/Cold/Warm/Hot/On Fire levels, comments-per-day calculation, empty comments
ResponseTypeIndicatorTest	8	Question/positive/debate/general detection, mixed types, null content, percentages
RecurringKeywordsIndicatorTest	6	Keyword extraction, stop-word filtering, HTML stripping, max 5 keywords
TopContributorsIndicatorTest	5	Top 3 ranking, anonymous handling, total contributor count
UniqueCommenterIndicatorTest	5	Unique count, diversity ratio, case-insensitive names, high/low diversity labels
TfIdfBreakdownStrategyTest	7	TF-IDF clustering, theme generation, representative comments, HTML handling, null/empty content
BreakdownStrategySelectorTest	8	Dynamic selection by comment count, manual strategy override, fallback behavior, available strategies
DiscussionOverviewTest	4	All 5 indicators registered, compute all, empty comments, error isolation

Run all Task 6 tests:

```
mvn test -pl app -Dtest="org.apache.roller.weblogger.business.pulse.ActivityLevelIndicatorTest,org.apache.roller.weblogger.business.pulse.ResponseTypeIndicatorTest,org.apache.roller.weblogger.business.pulse.RecurringKeywordsIndicatorTest,org.apache.roller.weblogger.business.pulse.TopContributorsIndicatorTest,org.apache.roller.weblogger.business.pulse.UniqueCommenterIndicatorTest,org.apache.roller.weblogger.business.pulse.TfIdfBreakdownStrategyTest,org.apache.roller.weblogger.business.pulse.BreakdownStrategySelectorTest,org.apache.roller.weblogger.business.pulse.DiscussionOverviewTest"
```

How to Use

Feature 1: User Highlights (Stars & Trending)

Star a weblog: 1. Log in to Apache Roller 2. Visit any blog page 3. Click “Star this blog” in the sidebar 4. Button changes to “Unstar this blog”

Star a blog entry: 1. Navigate to any individual blog post 2. Click “Star this post” below the entry 3. You are redirected back to the same entry

View starred content: - Starred Weblogs: `/roller-ui/starredWeblogs` - Starred Entries: `/roller-ui/starredEntries` (paginated)

View trending content: - Trending: `/roller-ui/trending` (top 5 weblogs and entries by star count)

Feature 2: Transforming Feeds Pipeline

The pipeline runs automatically when any blog entry is saved. No user action is required.

Admin configuration (in `roller.properties`): - Enable/disable each of the 5 steps independently - Configure reading speed (WPM) and max tags - AI summary uses the same Gemini API key as Community Pulse (`pulse.llm.apiKey`) - Changes take effect after application restart

Effects visible in published posts: - Profane words replaced with asterisks - AI-generated summary shown on blog list pages with “Read More” link to full text - Colored sentiment badge (Positive/Neutral/Negative) at the top of each entry - “X min read” badge indicating estimated reading time - Auto-generated tags appended at the bottom